Introduction

This exercise is designed to build muscle memory around one of the most commonly confused pairs of SQL operators. On paper, UNION and UNION ALL look nearly identical and same syntax, same idea of stacking two result sets together but they behave very differently in practice, and picking the wrong one can quietly corrupt a report. Ten questions will be answered using Databricks and the SQL and results of each questions will be posted.

Question 1: Combine the account holders from both branches into one unique list. Each person should appear only once, even if they hold accounts at both branches.

```
196  -- ============================================================================
197  -- SECTION 1 — QUESTIONS 01-05 (UNION)
198  -- ============================================================================
199
200  -- Q1: Unique account holders across both branches
201  SELECT account_id, account_holder, city FROM branch_sandton_accounts
202  UNION
203  SELECT account_id, account_holder, city FROM branch_rosebank_accounts
204  ORDER BY account_id;
```

dd parameter

Table +

	account_id	account_holder	city
1	A001	Nomvula Dlamini	Sandton
2	A002	David Mokoena	Sandton
3	A003	Lerato Sithole	Rosebank
4	A003	Lerato Sithole	Sandton
5	A004	Peter Nkosi	Rosebank
6	A004	Peter Nkosi	Sandton
7	A005	Zanele Khumalo	Rosebank
8	A006	Thabo Motha	Rosebank

Question 2: Build a unique product catalogue that shows every product the bank offers, without repeating any product that appears in both lists.

```
205
206  -- Q2: Unique product catalogue
207  SELECT product_code, product_name, product_type FROM savings_products
208  UNION
209  SELECT product_code, product_name, product_type FROM current_products
210  ORDER BY product_code;
211
```

dd parameter

Table +

	product_code	product_name	product_type
1	CR01	Standard Current	Current
2	CR02	Gold Current	Current
3	CR03	Business Current	Current
4	SV01	Basic Savings	Savings
5	SV02	Premium Savings	Savings
6	SV03	Youth Savings	Savings
7	SV04	Business Savings	Savings

Question 3 The HR team needs a unique list of all staff who work in either Retail Banking or Corporate Banking. Some staff are assigned to both. Each staff member should appear only once.

```
211
212  -- Q3: Unique staff across Retail + Corporate Banking
213  SELECT staff_id, staff_name, email FROM retail_banking_staff
214  UNION
215  SELECT staff_id, staff_name, email FROM corporate_banking_staff
216  ORDER BY staff_id;
217
218  -- Q4: Unique cities served (mobile + digital)
```

Add parameter

Table

	staff_id	staff_name	email
1	S001	Mpho Radebe	mpho.radebe@nexbank.co.za
2	S002	Brian Tshabalala	brian.tshabalala@nexbank.co.za
3	S003	Aisha Patel	aisha.patel@nexbank.co.za
4	S004	Kabelo Moabelo	kabelo.moabelo@nexbank.co....
5	S005	Nandi Dube	nandi.dube@nexbank.co.za
6	S006	Sipho Khumalo	sipho.khumalo@nexbank.co.za

Question 4: Marketing needs a unique list of all cities where NexBank serves customers — either through its mobile branches or its digital branch (which serves customers online regardless of city). No city should appear twice.

```
218  -- Q4: Unique cities served (mobile + digital)
219  SELECT city_code, city_name, region FROM mobile_branch_cities
220  UNION
221  SELECT city_code, city_name, region FROM digital_branch_cities
222  ORDER BY city_code;
```

Add parameter

Table

	city_code	city_name	region
1	C01	Johannesburg	Gauteng
2	C02	Pretoria	Gauteng
3	C03	Cape Town	Western Cape
4	C04	Durban	KwaZulu-Natal
5	C05	Polokwane	Limpopo
6	C06	Port Elizabeth	Eastern Cape

Question 5: The digital marketing team ran two campaigns last month — one via push notification and one via in-app banner. Build a unique list of every customer who was targeted. If a customer received both, they must appear only once.

```
223
224  -- Q5: Unique customers targeted (push + in-app banner)
225  SELECT customer_id, customer_name, segment FROM push_notification_targets
226  UNION
227  SELECT customer_id, customer_name, segment FROM inapp_banner_targets
228  ORDER BY customer_id;
229
```

Add parameter

Table

	customer_id	customer_name	segment
1	C1001	Nomsa Zwane	Retail
2	C1002	Andile Buthelezi	Retail
3	C1003	Fatima Mahomed	Premium
4	C1004	Ryno van Zyl	Retail
5	C1005	Thandeka Cele	Retail
6	C1006	Samuel Nkosi	Premium

Question 6: The fraud team needs the complete log of all ATM transactions from machine ATM-01 and machine ATM-02. Every transaction must appear — even if the same transaction appears in both logs due to a system sync error.

```
231  -- ============================================================================
232  -- SECTION 2 — QUESTIONS 06-10 (UNION ALL)
233  -- ============================================================================
234
235  -- Q6: Full ATM transaction log (duplicates kept, e.g. T1003)
236  SELECT txn_id AS transaction_id, account_id, amount, transaction_date FROM atm01_transactions
237  UNION ALL
238  SELECT txn_id AS transaction_id, account_id, amount, transaction_date FROM atm02_transactions
239  ORDER BY transaction_id;
240
241  -- Q7: Full loan application list (duplicates kept, e.g. LA003)
```

Add parameter

Table

	transaction_id	account_id	amount	transaction_date
1	T1001	A001	500.00	2025-01-05
2	T1002	A002	1200.00	2025-01-06
3	T1003	A001	300.00	2025-01-07
4	T1003	A001	300.00	2025-01-07
5	T1004	A003	750.00	2025-01-08
6	T1005	A004	900.00	2025-01-09
7	T1006	A002	450.00	2025-01-10
8	T1007	A005	150.00	2025-01-11

Question 7: The credit team wants a full list of all loan applications received from the Gauteng region and the Western Cape region. Every application must be included — duplicates are allowed because both regions may have captured the same application.

```
240
241  -- Q7: Full loan application list (duplicates kept, e.g. LA003)
242  SELECT app_id AS application_id, customer_id, loan_type, amount_requested FROM gauteng_loan_applications
243  UNION ALL
244  SELECT app_id AS application_id, customer_id, loan_type, amount_requested FROM western_cape_loan_applications
245  ORDER BY application_id;
```

Add parameter

Table

	application_id	customer_id	loan_type	amount_requested
1	LA001	C1001	Home Loan	850000.00
2	LA002	C1002	Vehicle Loan	220000.00
3	LA003	C1003	Personal	45000.00
4	LA003	C1003	Personal	45000.00
5	LA004	C1004	Home Loan	1200000.00
6	LA005	C1005	Vehicle Loan	310000.00
7	LA006	C1006	Business	900000.00
8	LA007	C1007	Home Loan	1500000.00

Question 8: The customer experience team needs a full record of every complaint logged this month, whether it came via email or via the mobile app. Every complaint must appear — the team needs all of them to calculate total complaint volumes.

```
247
248  -- Q8: Full complaint log (email + app), no dedup
249  SELECT complaint_id, customer_id, category, logged_date FROM email_complaints
250  UNION ALL
251  SELECT complaint_id, customer_id, category, logged_date FROM app_complaints
252  ORDER BY complaint_id;
```

Add parameter

Table

	complaint_id	customer_id	category	logged_date
1	AC001	C2005	App Bug	2025-02-05
2	AC002	C2001	Card Issue	2025-02-06
3	AC003	C2006	Fees	2025-02-07
4	AC004	C2007	Delay	2025-02-08
5	EC001	C2001	Card Issue	2025-02-01
6	EC002	C2002	App Bug	2025-02-02
7	EC003	C2003	Delay	2025-02-03
8	EC004	C2004	Fees	2025-02-04

Question 9: Finance needs to reconcile all payment records from April and May into one table for the half-year report. Every payment from both months must appear — the total row count matters for reconciliation.

```
253
254  -- Q9: Reconciled April + May payments, every row counted
255  SELECT payment_id, account_id, amount, payment_date FROM april_payments
256  UNION ALL
257  SELECT payment_id, account_id, amount, payment_date FROM may_payments
258  ORDER BY payment_date, payment_id;
259
```

Add parameter

Table +

	payment_id	account_id	amount	payment_date
1	PAY001	A001	12500.00	2025-04-05
2	PAY002	A002	4800.00	2025-04-10
3	PAY003	A003	9200.00	2025-04-15
4	PAY004	A004	3300.00	2025-04-20
5	PAY005	A001	12500.00	2025-05-05
6	PAY006	A005	7600.00	2025-05-10
7	PAY007	A002	5100.00	2025-05-15
8	PAY008	A006	2800.00	2025-05-20

Question 10 The accounting system records debits and credits in two separate tables. Combine all entries into one general ledger for the audit. Every single entry from both tables must appear — no rows should be dropped.

```
259
260  -- Q10: Combined general ledger (debits + credits), every entry kept
261  SELECT entry_id, account_id, entry_type, amount, entry_date FROM debit_entries
262  UNION ALL
263  SELECT entry_id, account_id, entry_type, amount, entry_date FROM credit_entries
264  ORDER BY entry_date, entry_id;
265  -- Expected result: 8 rows total (4 debit + 4 credit rows, nothing dropped).
```

Add parameter

Table +

	entry_id	account_id	entry_type	amount	entry_date
1	CR001	A001	Credit	500.00	2025-06-01
2	DR001	A001	Debit	500.00	2025-06-01
3	CR003	A002	Credit	1200.00	2025-06-02
4	DR002	A002	Debit	1200.00	2025-06-02
5	DR003	A003	Debit	750.00	2025-06-03
6	DR004	A004	Debit	300.00	2025-06-04
7	CR002	A005	Credit	900.00	2025-06-05
8	CR004	A006	Credit	150.00	2025-06-06

Bonus Questions

Question 1

They should use UNION. The whole point of a watch list is to know which customers are flagged as high-risk and not how many times they were flagged. If a customer appeared on both the January and February lists, that is still just one person the fraud team needs to keep an eye on, not two.

Question 2

They need UNION ALL. The task explicitly says to count every transaction even if the same transaction was recorded in two systems on the same day, which means duplicates are not noise here and they are re meaningful data points that need to be counted and probably investigated.

Question 3

The problem is a column mismatch. The first SELECT pulls two columns

SELECT account_id, account_holder from branch_sandton_accounts,

but the second SELECT pulls three columns

SELECT account_id, account_holder, city

FROM branch_rosebank_accounts.

To correct it, it is either city is added on the first SELECT or removed from the second SELECT.

Conclusion

The main code is posted onto the GitHub.